

Behavioral Design Patterns

Complete Study Guide – CSE-214

Shamsul Haque Sami
BUET, Student ID 2305055

August 29, 2026

Contents

1	What Are Behavioral Patterns?	2
2	The Big Picture: Pattern Recognition Map	2
2.1	Quick Reference Table	2
2.2	Pairwise Disambiguation (the pairs everyone mixes up)	3
3	Strategy Pattern	5
3.1	Structure	5
3.2	Canonical Example – Duck Simulator	5
4	Template Method Pattern	7
4.1	Structure	7
4.2	Canonical Example – CaffeineBeverage	7
5	Observer Pattern	9
5.1	Structure	9
5.2	Canonical Example – WeatherData	9
6	Mediator Pattern	11
6.1	Structure	11
6.2	Canonical Example – UI Dialog	11
7	State Pattern	13
7.1	Structure	13
7.2	Canonical Example – Gumball Machine	13
8	Practice Question Bank	15
9	Bonus: Mixed-Pattern Practice Problems	18
10	Final Cheat Sheet	19

1 What Are Behavioral Patterns?

Definition

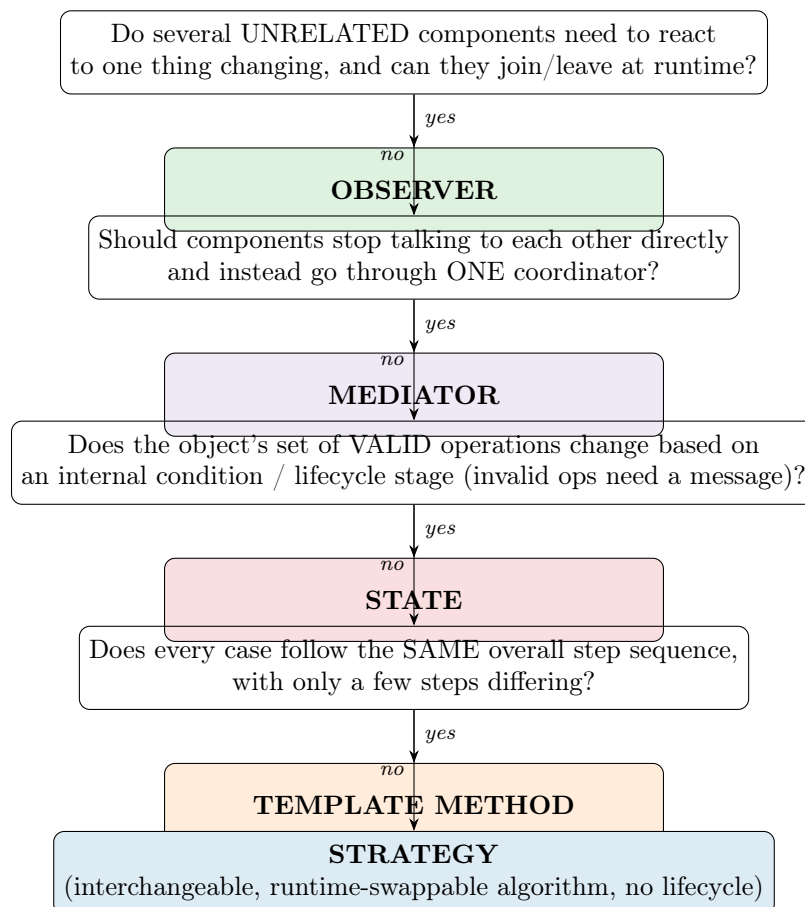
Behavioral patterns deal with how objects behave and interact — defining communication, responsibility flow, and how tasks are carried out between objects. They are concerned with **algorithms** and the **assignment of responsibilities** between objects.

The lab syllabus covers five behavioral patterns: **Strategy**, **Template Method**, **Observer**, **Mediator**, and **State**. (The lecture slide also covers *Command*, but it is not part of the syllabus, so it is intentionally left out of this guide.)

Every one of these patterns is really just answering the same underlying question in a different way: *“An object needs to behave differently in different circumstances – how do I organize that without a mess of if/else statements that grows forever?”* The five patterns are five different answers to that question, and telling them apart is 90% of the battle in a lab exam.

2 The Big Picture: Pattern Recognition Map

Read the problem statement and ask these questions, roughly in this order:



2.1 Quick Reference Table

Pattern	One-line idea	Trigger phrase in the question	Canonical example
Strategy	Encapsulate each algorithm as an object; hold & swap it via composition.	“switch between modes/plans at runtime”, “pluggable algorithm”	Duck Simulator (fly/quack behavior)
Template Method	Freeze the step order in the base class; let subclasses override a few steps.	“every X follows the same overall flow, but a few steps differ per Y”	CaffeineBeverage (Tea/Coffee)
Observer	Keep a list of listeners, notify them all when state changes; subscribe/unsubscribe at will.	“several groups react to one update”, “subscribe/unsubscribe”	WeatherData (displays)
Mediator	Route ALL inter-component communication through one coordinator.	“components shouldn’t talk directly”, “goes through a central X”	UI Dialog / Smart Home Hub
State	One class per condition; object delegates every operation to its current-state object.	“depending on its current condition/status, the same action means something different (or is invalid)”	Gumball Machine

2.2 Pairwise Disambiguation (the pairs everyone mixes up)

Strategy vs. Template Method

Both let you vary part of an algorithm. **Template Method** is based on *inheritance* – you alter parts of an algorithm by extending it in a subclass, and it works at the *class* level (static, chosen at compile time). **Strategy** is based on *composition* – you alter behavior by handing the object a different strategy object, and it works at the *object* level (dynamic, can change at runtime).

State vs. Strategy

Structurally these two are *identical twins* – both delegate work to a helper object via composition. The difference is *intent* and *awareness*: **Strategy** makes its helper objects completely independent and unaware of each other – the *client* usually picks which one to plug in. **State** does not restrict dependencies between concrete states – states are allowed to know about each other and trigger transitions themselves (`machine.setState(...)` called from *inside* a state class), because they represent stages of one single lifecycle.

Mediator vs. Observer

Mediator eliminates mutual dependencies – components depend on ONE mediator instead of each other, and the mediator usually *hard-codes* who talks to whom (an if/else chain). **Observer** establishes a dynamic, *one-way*, one-to-many connection – observers can subscribe/unsubscribe on their own, and the subject doesn't know or care what each observer does with the notification.

3 Strategy Pattern

Definition

Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it.

3.1 Structure

- **Strategy interface(s)** – one per independent behavior family (FlyBehavior, QuackBehavior).
- **Concrete Strategies** – each implements the interface with one specific algorithm (FlyWithWings, FlyNoWay, FlyRocketPowered).
- **Context** – *holds* a reference to a strategy object (composition, not inheritance) and *delegates* to it (Duck.performFly() calls flyBehavior.fly()).
- A **setter** on the context lets the strategy be swapped at runtime (setFlyBehavior(...)).

3.2 Canonical Example – Duck Simulator

```
1 interface FlyBehavior { void fly(); }
2 interface QuackBehavior { void quack(); }
3
4 class FlyWithWings implements FlyBehavior {
5     public void fly() { System.out.println("I'm flying!!"); }
6 }
7 class FlyNoWay implements FlyBehavior {
8     public void fly() { System.out.println("I can't fly"); }
9 }
10
11 abstract class Duck {
12     protected FlyBehavior flyBehavior;
13     protected QuackBehavior quackBehavior;
14
15     public void performFly() { flyBehavior.fly(); }
16     public void performQuack() { quackBehavior.quack(); }
17
18     // THE KEY MOVE: swap the algorithm object at runtime
19     public void setFlyBehavior(FlyBehavior fb) { this.flyBehavior = fb; }
20 }
21
22 class ModelDuck extends Duck {
23     public ModelDuck() { flyBehavior = new FlyNoWay(); }
24 }
25 // model.setFlyBehavior(new FlyRocketPowered());  <-- runtime swap
```

Full extended template: 1_StrategyPatternTemplate.java.

Recognize it when you see...

- The question lets the client pick between ≥ 2 interchangeable ways of doing ONE thing (sorting, routing, discounting, flying...).
- There is no notion of the object “moving through stages” – picking algorithm A today doesn’t restrict which algorithm you can pick tomorrow.
- Explicit phrase like “at runtime you should be able to change the ___ used”.

Exam Tips & Pitfalls

- Don’t inherit and override – *compose* a strategy field and delegate. If you catch yourself writing `extends` to change one small piece of behavior, you’re doing Template Method or plain bad inheritance, not Strategy.
- If there are TWO independent behavior axes (like fly *and* quack), that’s two separate strategy interfaces, not one bloated one.
- Constructors typically set a sensible default strategy; a setter allows changing it later.

4 Template Method Pattern

Definition

Template Method pattern defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure.

4.1 Structure

- **Template method** – ONE method holding the fixed step sequence, marked `final` so subclasses can never reorder it (`prepareRecipe()`).
- **Concrete steps** – identical for everyone, implemented in the base class (`boilWater()`, `pourInCup()`).
- **Abstract steps** – MUST differ per subclass, declared `abstract` (`brew()`, `addCondiments()`).
- **Hook methods** – MAY optionally differ; a concrete method with a default/do-nothing-ish body that a subclass can override (`customerWantsCondiments()`).

4.2 Canonical Example – CaffeineBeverage

```
1  abstract class CaffeineBeverage {
2      // THE TEMPLATE METHOD - fixed skeleton, final
3      public final void prepareRecipe() {
4          boilWater();
5          brew();
6          pourInCup();
7          if (customerWantsCondiments()) { // hook guards an optional step
8              addCondiments();
9          }
10     }
11     void boilWater() { System.out.println("Boiling water"); }
12     void pourInCup() { System.out.println("Pouring into cup"); }
13
14     abstract void brew();
15     abstract void addCondiments();
16
17     boolean customerWantsCondiments() { return true; } // HOOK
18 }
19
20 class Tea extends CaffeineBeverage {
21     void brew() { System.out.println("Steeping the tea"); }
22     void addCondiments() { System.out.println("Adding lemon"); }
23 }
```

Full extended template: `2_TemplateMethodPatternTemplate.java`.

Recognize it when you see...

- The question explicitly numbers a FIXED sequence of steps (“every visit/order/request follows steps 1..N in order”).
- Different “departments”/“types”/“categories” only customize a couple of those steps; the rest, and the ORDER, never change.
- Hospital-visit-style questions (Check-In → Vitals → Assessment → Treatment → Discharge) are the textbook shape.

Exam Tips & Pitfalls

- Mark the template method `final` (or at least say so in your design) – this is often explicitly checked.
- If a step is "sometimes skipped", that’s a hook (boolean-returning method), not a plain abstract method.
- Don’t be tempted to put an if/else inside the template method that branches on subclass type – that defeats the whole pattern. Branch only on a *hook’s return value*.

5 Observer Pattern

Definition

Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. (Publisher == Subject, Subscriber == Observer)

5.1 Structure

- **Subject interface** – registerObserver, removeObserver, notifyObservers.
- **Observer interface** – one method every listener implements, e.g. update(...).
- **Concrete Subject** – keeps a *list* of observers; loops over it inside notifyObservers() whenever its own state changes.
- **Concrete Observers** – register themselves with the subject (usually in their own constructor) and react independently inside update().

5.2 Canonical Example – WeatherData

```
1 interface Subject {
2     void registerObserver(Observer o);
3     void removeObserver(Observer o);
4     void notifyObservers();
5 }
6 interface Observer { void update(float temp, float humidity, float
7     pressure); }
8 class WeatherData implements Subject {
9     private final List<Observer> observers = new ArrayList<>();
10    public void registerObserver(Observer o) { observers.add(o); }
11    public void removeObserver(Observer o) { observers.remove(o); }
12    public void notifyObservers() {
13        for (Observer o : observers) o.update(temperature, humidity,
14            pressure);
15    }
16    public void measurementsChanged() { notifyObservers(); }
17 }
18 class CurrentConditionsDisplay implements Observer {
19     public CurrentConditionsDisplay(Subject weatherData) {
20         weatherData.registerObserver(this); // self-registers
21     }
22     public void update(float t, float h, float p) { /* ...display... */ }
23 }
```

Full extended template (also includes a generic MessageBoard/ Listener variant for single-payload, message-board-style problems): 3_ObserverPatternTemplate.java.

Recognize it when you see...

- Several *unrelated* components need to react the moment one thing changes – and they shouldn't need to know about each other.
- The problem explicitly allows joining/leaving at runtime (“groups can subscribe/unsubscribe”).
- Any “ticker / dashboard / notification-board” flavored problem (RavenBoard, stock ticker, weather displays).

Exam Tips & Pitfalls

- If the payload is a single value (a `String` message, a price) rather than several typed fields, it's fine (and cleaner) to make `Observer/Subject` generic or just use a `String`, e.g. `RavenBoard`, `StockData`.
- Remember to actually support `removeObserver` being called *mid-run* – don't hardcode the observer list at construction only.
- Each concrete observer typically reacts to only a *subset* of messages (Scouts only care about “Enemy” messages) – that filtering logic lives *inside* the observer's own `update()`, not in the subject.

6 Mediator Pattern

Definition

Mediator pattern restricts direct communications between objects and forces them to collaborate only via a mediator. Lets you reduce chaotic dependencies between objects.

6.1 Structure

- **Mediator interface** – one method components call to report an event, e.g. `notify(sender, event)`.
- **Component base class** – every element holds a reference to the mediator (passed via constructor) and, instead of talking to other components directly, calls `mediator.notify(this, "event")`.
- **Concrete Mediator** – knows about ALL the components and contains the actual coordination logic (an if/else chain on sender + event). This is the *only* class that changes when the coordination rules change.

6.2 Canonical Example – UI Dialog

```
1 interface Mediator { void notify(Component sender, String event); }
2
3 abstract class Component {
4     protected Mediator mediator;
5     public Component(Mediator mediator) { this.mediator = mediator; }
6 }
7
8 class Button extends Component {
9     public Button(Mediator mediator) { super(mediator); }
10    public void click() { mediator.notify(this, "click"); }
11 }
12
13 class AuthenticationDialog implements Mediator {
14     private Button okBtn;
15     private Checkbox businessCheckbox;
16     // ... construct all components, passing "this" as mediator ...
17
18     public void notify(Component sender, String event) {
19         if (sender == businessCheckbox && event.equals("check")) {
20             // enable/disable the company-name field
21         } else if (sender == okBtn && event.equals("click")) {
22             // validate everything, then close
23         }
24     }
25 }
```

Full extended template (also includes a Smart-Home-Hub style sensor/blinds/AC example): `4_MediatorPatternTem`

Recognize it when you see...

- The question says explicitly “components should not talk to each other directly”.
- There’s a single obvious “controller” / “hub” / “dialog” / “tower” that everything reports to.
- Coordination logic is centralized and *fixed* – not something that changes dynamically at runtime (that would drift toward Observer instead).

Exam Tips & Pitfalls

- Watch out for constructor circular-dependency: if the mediator needs references to components AND those components need the mediator, wire one side with a setter after construction (see the Smart-Home-Hub demo in the template) rather than trying to pass everything through constructors simultaneously.
- Keep components “dumb” – they only know how to call `notify()`; ALL of the “what happens next” logic belongs in the concrete mediator.
- If in the actual problem, the “mediator’s” behavior should also be pluggable/swappable at runtime, that’s really Mediator + Strategy combined (see §9).

7 State Pattern

Definition

Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

This is the pattern to lean on hardest for “lifecycle / workflow” problems (order processing, subscription tiers, return & refund workflows, document approval, etc.) – it is by far the most common shape in past behavioral-pattern papers.

7.1 Structure

- **State interface** – ONE method per possible action/operation the system supports.
- **Context** – holds a reference to the CURRENT state object (plus a field per possible state); every public operation just *delegates* to the current state. The context itself contains *no* if/else on what state it’s in.
- **Concrete States** – one class per condition. Each implements every operation:
 - if valid here → do the real work and call `context.setState(...)` to transition;
 - if *not* valid here → just print/return an appropriate message instead of performing it.
- Adding a new condition later = add one new class; nothing else changes (Open/Closed Principle).

7.2 Canonical Example – Gumball Machine

```
1 interface State {
2     void insertQuarter();
3     void ejectQuarter();
4     void turnCrank();
5     void dispense();
6 }
7
8 class GumballMachine {
9     private State state; // current state -- delegate everything to it
10    public void insertQuarter() { state.insertQuarter(); }
11    public void turnCrank()     { state.turnCrank(); state.dispense(); }
12    void setState(State state) { this.state = state; }
13 }
14
15 class NoQuarterState implements State {
16     private final GumballMachine machine;
17     public void insertQuarter() {
18         System.out.println("You inserted a quarter");
19         machine.setState(machine.getHasQuarterState()); // valid ->
20         transition
21     }
22     public void ejectQuarter() { System.out.println("You haven't inserted
23         a quarter"); } // invalid
24     public void turnCrank()    { System.out.println("You turned, but
25         there's no quarter"); } // invalid
```

```

23     public void dispense()        { System.out.println("You need to pay
      first"); }                  // invalid
24 }

```

Full extended template (all five states, including `WinnerState`): `5_StatePatternTemplate.java`.

Recognize it when you see...

- The question lists named *conditions/statuses* (Requested / Approved / Delivered / ... , or Common / Plus / Lux) and, for each, tells you which operations are allowed and where they lead.
- Explicit language like “for any operation not valid at this point, produce an appropriate message rather than performing it.”
- Some conditions are “final” – no operation changes them further.

Exam Tips & Pitfalls

- Give the context getters for each state instance (`getNoQuarterState()`, etc.) so concrete states can reference each other for transitions without creating new objects every time.
- “Temporarily switch to state X, then automatically return to whichever state was active before” (like the subscription tiers’ Lux-activation) is solved by simply *remembering the previous state* in a field before switching, then restoring it later – no new pattern needed, just a bit of extra bookkeeping in the context or in the temporary state itself.
- Promote/demote-style operations (tier upgrades) are just ANOTHER pair of methods on the `State` interface, with each concrete state’s implementation calling `setState()` on the next/previous tier – same recipe as `insertQuarter()`.
- Don’t forget: unlike Strategy, concrete State classes are allowed to know about each other and about the context – that’s normal and expected here.

8 Practice Question Bank

The questions below are drawn from your previous-year multi-section paper and last week's paper for another section. Try to (1) name the pattern, (2) sketch the class/interface list, and (3) note the tricky bit *before* reading the “Approach” box.

Q1 – Subscription Tiers (State)

[Section A1, previous year] A monthly brain-support subscription has three tiers – Common, Plus, Lux – each with a coverage radius (Common: 0–10 km; Plus/Lux: 0–50 km). Operations: `travelCheck(km)`, `activateLux(hours)`, `setMood(...)`, `promote()`, `demote()`. `travelCheck` prints STABLE/UNSTABLE based on the current tier's range (an alert + eventual `travelCheck(0)` on going out of range). `activateLux(hours)` temporarily switches to Lux, then automatically reverts to whichever tier (Common/Plus) was active before. `setMood(...)` only works while Lux is active.

Pattern: State (tiers = concrete states) with a small “remember-and-restore” twist for `activateLux`.

- **State interface:** `travelCheck(km)`, `setMood(mood)`, `promote()`, `demote()`, plus a way to expose the tier's safe range.
- **Concrete states:** `CommonState`, `PlusState`, `LuxState` – `promote()/demote()` on each just call `context.setState(...)` to the neighboring tier (or itself, for the two “no change” edge cases).
- `setMood(...)` is a no-op printing “Mood control unavailable.” in `CommonState/PlusState`, and does the real thing only in `LuxState`.
- `activateLux(hours)`: context stores its *current* state in a temp field, calls `setState(luxState)`, sleeps, then calls `setState(rememberedState)`. This bookkeeping can live in the context (simplest) rather than needing a whole new pattern.

Q2 – RavenBoard (Observer)

[Section A2, previous year] A message board where ravens deliver scrolls (e.g. “Enemy spotted near the river”). Multiple groups (Commander, Scouts, Supply Team) receive new messages and act on them (Scouts print “Dispatch riders!”, Supply Team prints “Update inventory!”). Groups can subscribe/unsubscribe at runtime.

Pattern: Observer, single-String-payload variant.

- **Subject** = `RavenBoard` with `subscribe/unsubscribe/postMessage(String)`.
- **Observer** = `Listener` interface with `onMessage(String)`; each concrete listener (Scouts, SupplyTeam, Commander) filters for keywords relevant to it *inside* its own `onMessage`.
- Demonstrate unsubscribe by removing a listener mid-run and showing it no longer reacts to the next post.

Q3 – Smart Home Hub (Mediator)

[Section B2, previous year] A Light Sensor, Automatic Blinds, and an Air Conditioner should not talk to each other directly – they all report to a Central Hub. High Brightness → Hub tells Blinds to close. Blinds close → Hub tells AC to turn on.

Pattern: Mediator.

- HubMediator interface with `notify(sender, event)`.
- LightSensor, Blinds, AirConditioner each just call `hub.notify(this, ...)` – none of them know about each other.
- CentralHub contains the two if-branches that chain the reactions. Watch the wiring order (hub needs blinds, blinds needs hub) – use a setter to break the circular-constructor problem.

Q4 – Hospital Visit Simulator (Template Method)

[Section C1, previous year] Every visit follows: Check-In → Record Vitals → Assessment → Treatment → Discharge Summary, printed in order. General/Pediatrics/Emergency departments customize only Assessment and Treatment.

Pattern: Template Method.

- `Visit.processVisit()` (final) calls `checkIn()`, `recordVitals()`, `assessment()`, `treatment()`, `dischargeSummary()` in that fixed order.
- `checkIn`, `recordVitals`, `dischargeSummary` are concrete (identical for all departments).
- `assessment()`, `treatment()` are abstract – `GeneralDepartment`, `PediatricsDepartment`, `EmergencyDepartment` each implement them differently.

Q5 – Stock Dashboard (Observer)

[Section C2, previous year] A StockData feed notifies a Ticker Tape, a Graph widget, and a Buy/Sell Bot whenever a price changes. The feed shouldn't know the specifics of the widgets; widgets can be added or removed.

Pattern: Observer (near-identical shape to WeatherData, just a single price field instead of three).

- StockData implements Subject; TickerTape, Graph, BuySellBot implement Observer.
- BuySellBot's `update(price)` contains its own trade-trigger logic – the subject doesn't need to know that logic exists.

Q6 – E-commerce Return & Refund Workflow (State)

[Held last week, another section, this year] A return request moves through Requested → Approved → Delivered → Processing Refund → {Refunded / Rejected / Cancelled}. Operations: `updateReason(reason)`, `approve()`, `reject()`, `cancel()`, `itemDelivered()`, `inspect(eligible)`, `refundSuccessful()`, `refundFailed()`. Invalid operations at a given point should just produce a message.

Pattern: State – this is the single cleanest “textbook” State question in the whole set; it maps onto `GumballMachine` almost field-for-field.

- **ReturnState** interface: all eight operations above, each concrete state overrides only the ones that are valid there and leaves the rest printing an invalid-operation message (the default behavior can even live in a shared abstract base class to avoid repeating “operation not allowed” eight times per state).
- Concrete states: `RequestedState`, `ApprovedState`, `DeliveredState`, `ProcessingRefundState`, and the three terminal ones `RefundedState`/`RejectedState`/`CancelledState` (which override nothing further – everything is invalid there).
- `inspect(boolean eligible)` in `DeliveredState` branches internally: `eligible` → `setState(processingRefundState)` otherwise → `setState(rejectedState)`. This single method containing a branch that leads to two different transitions is a nice small twist worth practicing – compare it to `HasQuarterState.turnCrank()`’s random-winner branch in the Gumball Machine.
- `refundFailed()` in `ProcessingRefundState` should loop back to itself (“may be attempted again later”) rather than transitioning away – i.e. it’s a valid operation whose transition target is the *same* state.

9 Bonus: Mixed-Pattern Practice Problems

None of the source papers combine two patterns in one problem, but a lab exam occasionally will. These three are **not** from an official paper – I constructed them as extra practice for combining two patterns in a single design.

Bonus 1 – Music Player (Strategy + Observer)

A music player can play tracks using different, swappable *playback strategies* (Normal, Bass-Boost, LoFi – each processes audio differently) chosen at runtime. Independently, several unrelated UI widgets (`NowPlayingBar`, `PlaylistPanel`, `VolumeIndicator`) must all update automatically whenever the track changes, and widgets can be attached/detached while the app runs.

Hint: these are two completely independent concerns living in the same `Player` class – a `PlaybackStrategy` field (Strategy) that the player delegates `play()` to, PLUS a separate `List<Observer>` (Observer) that the player notifies on `trackChanged()`. Don't merge them into one interface; they solve different problems and should stay decoupled from each other.

Bonus 2 – Elevator Control System (State + Mediator)

A building has three elevator cars, each following its own lifecycle: `Idle` → `Moving` → `DoorOpen` → `Idle` (with the usual "invalid operation while doors are open" rules). Cars must never coordinate directly with each other; instead, a Central Dispatcher decides which idle car responds to a floor call button and tells that car to move.

Hint: each `ElevatorCar` is a State-pattern context on its own (its own `State` field/interface for `Idle`/`Moving`/`DoorOpen`). Separately, the `Dispatcher` is a Mediator that the floor-call buttons and the cars report to – `car.notify(dispatcher, "idle")` lets the dispatcher know which cars it may assign next. The two patterns touch at exactly one point: the dispatcher calls a method on whichever car it selects, and that call is what triggers that car's own internal `Idle` → `Moving` transition.

Bonus 3 – Document Review Workflow (Template Method + State)

Every document submitted for review follows the same fixed steps: `Validate Format` → `Run Automated Checks` → `Human Review` → `Record Decision`, always in that order. Separately, a document's status (`Draft` / `InReview` / `Approved` / `Rejected`) determines which of those actions are even allowed to be triggered right now – e.g. you cannot run "Human Review" on a document that is still `Draft`.

Hint: the four-step `processReview()` skeleton is a Template Method (fixed order, a couple of steps – e.g. the automated checks – differing per document *type*, like PDF vs. spreadsheet). Layered UNDERNEATH that, the document's `Draft/InReview/...` lifecycle is a State machine: each step of the template method first asks the document's *current state object* whether that action is allowed before doing the work, and the state object is what actually calls `setState(...)` to advance the lifecycle. Template Method controls the "in what order do things happen", while State controls "is this particular thing allowed to happen right now".

10 Final Cheat Sheet

Pattern	Roles	Signal in the question	Confusable with
Strategy	Strategy interface, Concrete Strategies, Context (composes & delegates)	“pluggable / swappable algorithm at runtime”	Template Method (composition vs. inheritance)
Template Method	Abstract class with final template method, concrete/abstract/hook steps	“same steps in order, a few differ per type”	Strategy (inheritance vs. composition)
Observer	Subject, Observer, concrete subject (list of observers), concrete observers (self-register)	“several react to one change”, “subscribe/unsubscribe”	Mediator (one-way broadcast vs. central coordinator)
Mediator	Mediator interface, Component base, Concrete Mediator (holds all the logic)	“must not talk directly, go through a hub/controller”	Observer (fixed hard-coded routing vs. dynamic broadcast)
State	State interface (1 method/action), Context (delegates, no if/else), Concrete States (valid → transition, invalid → message)	“current condition/status determines what’s allowed”, work-flows/lifecycles	Strategy (self-transitioning & state-aware vs. independent & unaware)

Companion files: *1_StrategyPatternTemplate.java*, *2_TemplateMethodPatternTemplate.java*, *3_ObserverPatternTemplate.java*, *4_MediatorPatternTemplate.java*, *5_StatePatternTemplate.java*
 – all compiled and tested with OpenJDK 21.